

Code Explanation

Data Processing Test Suite Explanation

This is a PySpark test suite that checks the data cleaning, joining, and aggregation logic. It uses Pytest fixtures to create sample customer and order data.

Overview

The code defines two fixtures: `sample\_customer\_data` and `sample\_order\_data`, which create sample DataFrames for customer and order data, respectively. It then defines two test functions: `test\_data\_cleaning` and `test\_join\_and\_aggregation`, which test the data cleaning and joining/aggregation logic.

Step 1: Creating a SparkSession Fixture

The first step is to create a SparkSession fixture using Pytest. This fixture is used to create a SparkSession that can be used throughout the test suite.

```
@pytest.fixture(scope="session")
def spark():
    """Create a SparkSession for testing"""
    return SparkSession.builder
        .appName("TestCustomerOrderProcessing")
        .master("local[*]")
        .config("spark.sql.warehouse.dir", tempfile.mkdtemp())
        .config("spark.sql.extensions", "io.delta.sql.DeltaSparkSessionExtension")
        .config("spark.sql.catalog.spark_catalog", "org.apache.spark.sql.delta.catalog.DeltaCatalog")
        .getOrCreate()
```

Step 2: Creating Sample Customer Data Fixture

The next step is to create a fixture for sample customer data.

```
@pytest.fixture
def sample_customer_data(spark):
    """Create sample customer data for testing"""
    schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        (None, "Invalid Customer", "invalid@example.com", "West"),
        ("C004", None, "no_name@example.com", "North"),
        ("C001", "John Doe", "john@example.com", "North"),
```

```
]
return spark.createDataFrame(data, schema)
```

Step 3: Creating Sample Order Data Fixture

Similarly, we create a fixture for sample order data.

```
@pytest.fixture
def sample_order_data(spark):
    """Create sample order data for testing"""
    schema = StructType([
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
        StructField("Date", DateType(), True),
        StructField("CustId", StringType(), True)
    ])

    data = [
        ("0001", "Item1", 10.0, 2, date(2023, 1, 15), "C001"),
        ("0002", "Item2", 15.0, 1, date(2023, 1, 20), "C002"),
        ("0003", "Item3", 20.0, 3, date(2023, 1, 25), "C003"),
        ("0004", "Item4", 25.0, 2, date(2023, 1, 30), "C001"),
        (None, "Invalid Order", 5.0, 1, date(2023, 2, 1), "C002"),
        ("0005", "Item5", None, 2, date(2023, 2, 5), "C003"),
        ("0001", "Item1", 10.0, 2, date(2023, 1, 15), "C001"),
    ]

    return spark.createDataFrame(data, schema)
```

Step 4: Testing Data Cleaning Logic

The `test\_data\_cleaning` function tests the data cleaning logic by cleaning the customer and order data, and then verifying the results.

```
def test_data_cleaning(spark, sample_customer_data, sample_order_data):
    """Test data cleaning logic"""
    clean_customer = (
        sample_customer_data
        .dropDuplicates()
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
    )

    clean_order = (
        sample_order_data
```

```

        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates()
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
    )

    assert clean_customer.count() == 3
    assert clean_order.count() == 4

    order_with_total = clean_order.filter(F.col("OrderId") == "0001")
    .first()
    assert order_with_total["TotalAmount"] == 20.0

```

Step 5: Testing Join and Aggregation Logic

The `test\_join\_and\_aggregation` function tests the join and aggregation logic by joining the cleaned customer and order data, and then aggregating the results.

```

def test_join_and_aggregation(spark, sample_customer_data, sample_order_data):
    """Test join and aggregation logic"""
    clean_customer = (
        sample_customer_data
        .dropDuplicates()
        .filter(
            (F.col("CustId").isNotNull()) &
            (F.col("Name").isNotNull()) &
            (F.col("EmailId").isNotNull()) &
            (F.col("Region").isNotNull())
        )
    )

    clean_order = (
        sample_order_data
        .withColumn("TotalAmount", F.col("PricePerUnit") * F.col("Qty"))
        .dropDuplicates()
        .filter(
            (F.col("OrderId").isNotNull()) &
            (F.col("ItemName").isNotNull()) &
            (F.col("PricePerUnit").isNotNull()) &
            (F.col("Qty").isNotNull()) &
            (F.col("Date").isNotNull()) &
            (F.col("CustId").isNotNull())
        )
    )

    joined_df = (
        clean_order

```

```

        .join(
            clean_customer,
            on="CustId",
            how="inner"
        )
        .select(
            "CustId", "Name", "EmailId", "Region", "OrderId",
            "ItemName", "PricePerUnit", "Qty", "Date", "TotalAmount"
        )
    )
    )

agg_df = (
    joined_df
    .groupBy("Name", "Date")
    .agg(F.sum("TotalAmount").alias("TotalAmount"))
)

assert joined_df.count() == 4

john_orders = agg_df.filter(F.col("Name") == "John Doe").collect
()
assert len(john_orders) == 2

john_jan15 = next((row for row in john_orders if row["Date"] ==
date(2023, 1, 15)), None)
assert john_jan15 is not None
assert john_jan15["TotalAmount"] == 20.0

```